API's

14 July 2026 12:02 PM

API (Application Programming Interface) :
An API is a contract that defines how different software systems communicate securely without exposing their internal implementation.

- It defines:
  - What requests can be made.
  - What data should be sent.
  - What response will be returned.
  - What errors may occur.
- Think of an **API as a receptionist/front desk**:
  - Receives requests.
  - Processes them.
  - Returns the appropriate response.
  - Hides the internal system.
- Examples:
  - Weather API → Returns weather data.
  - Payment API → Processes payments.
  - AI API → Generates AI responses.
- A good API also:
  - Validates input.
  - Authenticates users.
  - Authorizes access.
  - Rate limits requests.
  - Protects internal services.

1. What an API Contract Defines

An **API Contract** is the agreement between the **client (caller)** and the **server (provider)**. It clearly defines how they communicate.

It answers these questions:

Question	Example
What can the caller do?	GET /v1/orders/123
What must be sent?	Path, query, headers, request body
What is returned?	JSON, file, stream
What errors can occur?	Status codes (404, 500), error messages
Who is calling?	API Key, OAuth Token
What can they access?	Authorized resources only
What are the limits?	Rate limits, request size, timeout
How are changes handled?	API versioning (v1, v2)

Eg: API Contract in practice

Here is a simple API request to start a model run. It shows the method and path, an auth token, the content type, an idempotency key, and a JSON body. An idempotency key is a duplicate-protection key; we will come back to it later.

POST /v1/model-runs HTTP/1.1
Host: api.example.com
Authorization: Bearer <token>
Content-Type: application/json
Idempotency-Key: 5d4b6b3e-6b81-4e5a-8b7f-7d4c2d8fd831

API Request Contains

- HTTP Method (GET, POST, PUT, DELETE)
- URL/Endpoint
- Headers (Authorization, Content-Type)
- Request Body (JSON)
- Optional **Idempotency Key** (prevents duplicate requests)

API Response Contains

- Status Code (200, 201, 404, 500, etc.)
- Response Body (usually JSON)
- Error Message (if request fails)

2. APIs Are Boundaries

An **API acts as a boundary** between the client and the internal system. Clients interact only with the API, **not with the database or internal services**.

Why is this important?

- **Hides internals** – Clients don't know how the system works.
- **Allows safe changes** – Internal implementation can change without affecting clients.
- **Improves security** – No direct access to databases or services.
- **Clear ownership** – Different teams can manage different APIs.

- ♻ **Reusability** – The same API can serve web, mobile, and partner applications.
- 📊 **Easy monitoring** – Requests can be logged, traced, and audited.

APIs also create responsibility. If a public API breaks, customers may break. If an internal API has no clear owner, every change becomes a meeting. If a model API hides its latency, cost, or safety behavior, the systems calling it become harder to debug.

3. Types of APIs

APIs can be grouped by who uses them and how they are called.

Type	Used By	Example
🌍 **Public API**	External developers, customers	Google Maps API, Stripe API
🤝 **Partner API**	Selected business partners	Supplier or vendor APIs
🏢 **Internal API**	Teams within the same company	Payment Service → Inventory Service
📚 **Library API**	Code within the same application	Java Collections, Python list.append()

1. Public API

- Available to external users.
- Requires:
  - Authentication
  - Authorization
  - Rate limiting
  - Good documentation
  - Stable versions

Examples: Payment APIs, Weather APIs, AI APIs.

2. Partner API

- Shared only with trusted partners.
- Includes:
  - Partner-specific access
  - Audit logs
  - Data-sharing rules
  - Planned updates

Example: Amazon sharing APIs with delivery partners.

3. Internal API

- Used only inside an organization.
- Connects internal services.
- Still needs:
  - Documentation
  - Ownership
  - Error handling
  - Timeouts

Example:

```
Checkout Service
   |
   ├── Payment Service
   ├── Inventory Service
   ├── Notification Service
   └── Analytics Service
```

4. Library API

- Used within the same program through functions or classes.
- Hides internal implementation.

Example (Python):

```
numbers.append(10)
```

You use append() without knowing how the list is stored internally.

4. Network APIs

Most system design discussions focus on network APIs. A client sends data over a network to a server. The server responds right away, streams results, or starts work that can be checked later.

Request Parts

An HTTP API request usually has:

Part	Purpose	Example
Method	What kind of action this is	GET, POST, PATCH, DELETE
Path	Target resource or action	/v1/orders/ord_123
Query string	Filtering, pagination, options	?limit=50&cursor=abc
Headers	Extra information and credentials	Authorization, Accept, Content-Type
Body	Structured input	JSON request body

Those parts combine into a request on the wire:

```
GET /v1/orders?limit=20&status=paid HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer <token>
```

Response Parts

An HTTP API response usually has:

Part	Purpose	Example
Status code	Outcome at the HTTP level	200, 201, 400, 401, 404, 429, 500
Headers	Extra information about the response	Content-Type, Cache-Control, RateLimit-Remaining
Body	The returned data or error	JSON object, file bytes, stream

A successful response to the request above carries the matching body:

```
{
  "data": [
    {
      "id": "ord_123",
      "status": "paid",
      "total_cents": 8997
    }
  ],
  "next_cursor": "cur_456"
}
```

The body should not be the only place that says whether the request worked. A failed request usually should not return `200 OK` with `{ "success": false }`.

Status codes matter because clients, gateways, proxies, SDKs, and monitoring tools use them to understand what happened

5. Common API Styles

Different API styles are used based on the application's needs.

API Style	Description	Common Use
REST	Resource-based, uses HTTP methods	Web APIs, Mobile apps
GraphQL	Client requests only needed data	Complex UIs, Dashboards
gRPC	Fast service-to-service communication using Protocol Buffers	Microservices
WebSocket	Two-way, real-time communication	Chat, Games, Live Collaboration
Server-Sent Events (SSE)	One-way server → client stream	Notifications, AI token streaming
Webhooks	Server sends HTTP callbacks on events	Payment notifications, GitHub events
Message-Driven APIs	Communicate through message brokers	Background jobs, Event processing
SOAP	XML-based protocol	Legacy enterprise systems

When to Use Which?

- **REST** → Standard CRUD APIs (most common).
- **GraphQL** → When clients need flexible data.
- **gRPC** → Fast internal microservice communication.
- **WebSocket** → Real-time bidirectional communication.
- **SSE** → Continuous server updates (one-way).
- **Webhooks** → Notify another system when an event occurs.
- **Message-Driven** → Asynchronous workflows.
- **SOAP** → Older enterprise applications.

6. Authentication, Authorization, and Limits

An API usually needs to answer three basic questions:

1. Who is calling?
2. What are they allowed to do?
3. How much traffic are they allowed to send?

Authentication (Who are you?)

Verifies the identity of the caller.

Common Methods:

- API Key
- OAuth 2.0 Token
- Session Cookie
- mTLS (Mutual TLS)

Never send API keys or tokens in the URL (query parameters). Use the **Authorization** header.

Authorization (What can you do?)

Checks whether the authenticated user has permission to perform an action.
Example:

- ☑ User can view **their own** orders.
- ❌ User cannot view **another user's** orders.
  Authentication = Identity
  Authorization = Permissions

Rate Limits & Quotas

Protect the API from abuse and control usage.
Examples:

- **Rate Limit:** 100 requests/minute.
- **Quota:** 10,000 requests/month.

For AI APIs, limits may depend on:

- Tokens
- File size
- Model used
- Parallel requests

7. API Reliability

Production APIs must be clear about failure. Clients need to know when to wait, when to retry, and when to stop and show an error.

Timeouts

- Every API request should have a timeout.
- Prevents waiting forever if the server is slow.

Retries

Retry **only when it's safe**.
☑ Safe:

GET /orders/123
❌ Risky:

POST /payments
Retrying may charge the customer twice.

Idempotency

Sending the **same request multiple times** should produce the **same result**.
Used for:

- Payments
- Order creation
- Job submission
- AI model runs

Example: Use an **Idempotency-Key** to prevent duplicate processing.

Debugging and Observability

APIs should record enough information to debug production issues. At minimum, that usually means:

- A request ID or trace ID.
- The caller or tenant, when safe to log.
- The route and method.
- The status code.
- How long the request took.
- The request and response size.
- Whether the request was rate limited.
- Time spent calling downstream services.

Logs must not casually store secrets, credentials, raw access tokens, sensitive personal data, model prompts, or private documents. If a system logs sensitive data, it needs explicit controls for who can access it and how long it is kept.

8. How Engineers Use an API

Typical API integration steps:

1. 📖 Read the API documentation.
2. 🔐 Configure authentication.
3. 🧪 Test using curl, Postman, or an SDK.
4. ☑ Handle success and error responses.
5. ⏱ Add timeouts and retries.
6. 🔁 Use idempotency where needed.
7. 🚦 Handle rate limits and pagination.
8. 📋 Log request IDs and errors.
9. 🔒 Store API keys securely (environment variables, **not** source code or URLs).

Example C# request:

```
using System;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        // Read API token from environment variable
        string token = Environment.GetEnvironmentVariable("API_TOKEN");

        using HttpClient client = new HttpClient();

        // Set timeout
        client.Timeout = TimeSpan.FromSeconds(5);

        // Set headers
        client.DefaultRequestHeaders.Accept.Add(
            new MediaTypeWithQualityHeaderValue("application/json"));
        client.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue("Bearer", token);

        // API URL with query parameter
        string url = "https://api.example.com/v1/weather?city=London";

        HttpResponseMessage response = await client.GetAsync(url);

        if (response.StatusCode == System.Net.HttpStatusCode.OK)
        {
            string json = await response.Content.ReadAsStringAsync();
            Console.WriteLine(json);
        }
        else if (response.StatusCode == System.Net.HttpStatusCode.TooManyRequests)
        {
            throw new Exception("Rate limit exceeded");
        }
        else
        {
            throw new Exception($"API call failed: {(int)response.StatusCode}");
        }
    }
}
```